



University of Cape Town

ECO5040W — Financial Software Engineering

Business and Technical Specification

XRPL-Based FX Remittance Platform using RLUSD

Annita Ngoma Karabo Tigedi Kerry-Lynn Whyte

Liltha Mzamo Nikola Milosavljevic

Group 3

7 September 2026

This specification describes **Group 3's** XRPL remittance prototype as implemented in this GitHub fork (Karabo22Tigedi / FSE-Project). An earlier GitHub sketch by Kerry-Lynn Whyte established FastAPI, SQLite, Redis Streams, and Testnet Payments. This document is about *our* patched system, not that sketch. Cash-in and cash-out remain simulated. Using a dollar stablecoin does not remove South African exchange-control, anti-money-laundering, or licensing duties (University of Cape Town, 2026a). One hundred and twenty-nine automated tests currently pass. The examiner-facing product is the Vite React UI in **frontend/** (Section 15), talking to FastAPI on port 8000.

1 Business problem

Cross-border remittances move household income to people who often collect cash at an agent. Western Union and MoneyGram are the familiar brands: the sender pays rand at an agent or online, the operator takes a fee, and the recipient collects cash on the other side after showing identification. Those operators are large, licensed money transmitters with agent networks in many countries. They are also expensive on small sends, and the price the customer sees is often a mix of a posted fee and a worse-than-mid-market exchange rate.

The World Bank's Remittance Prices Worldwide (RPW) survey is the standard public comparison of corridor costs (World Bank, 2025). South Africa is a high-cost sending market on that survey. Rateweb's 2026 reading of the 2025 Q1 RPW release reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed send of about R1,370 (Rateweb, 2026). That is a fee of roughly 15–16% of the send amount before any FX spread. For a family sending grocery money, that is a large bite.

Cost is not the only friction. The IMF's diagnostic of the South Africa–Zimbabwe corridor describes cash-heavy, agent-based payouts, delays, and weak price transparency (International Monetary Fund, 2025). FinMark Trust's 2024 assessment of South Africa to the rest of SADC

likewise finds that cash collection and agent processes still dominate, and that customers often cannot see the full cost until they are already at the counter (FinMark Trust, 2024). Western Union and MoneyGram solve a real problem—getting cash to someone without a bank account—but they do it with many intermediaries, posted-plus-spread pricing, and a payout model that is slow to digitise.

Stablecoins such as RLUSD (a US-dollar token issued on the XRP Ledger) can move value between wallets in seconds once both sides hold the token. That does not magically replace the cash-in door in Johannesburg or the cash-out door in Harare. The brief is explicit: a prototype may *simulate* those fiat legs (University of Cape Town, 2026a). Our platform therefore uses UCTUSD (Marc’s course IOU) as the current *settlement* asset between a confirmed rand payment and a custodial recipient wallet; the API slot remains `rlusd_amount` and can later point at official RLUSD via `.env` if funded. We do not claim that putting value on a ledger deletes the South African Reserve Bank, FICA, or a remittance licence.

What we do claim, as a design, is narrower. A quoted fee of a fixed rand amount plus a percentage, plus a published FX margin on the conversion rate, is easier to audit than an opaque corridor tariff. A 15-minute quote with an explicit cancel path stops abandoned drafts from consuming a sender’s daily limit. An on-chain Payment with a stored transaction hash gives the recipient something they can look up on a Testnet explorer. Those are the product bets this specification implements.

2 User journey

The journey follows the brief (University of Cape Town, 2026a) and is implemented end-to-end on the API and on the web UI in Section 15.

1. A customer registers (`POST /auth/register`) and logs in (`POST /auth/login`). The same customer account can send or receive; role is `customer` or `admin`, not two person types.
2. The sender submits mock KYC (`POST /kyc`). An admin approves or rejects it. Unapproved senders cannot create quotes.
3. The sender adds a beneficiary (`POST /beneficiaries`). If mobile or email already matches a registered user, the record links immediately; if the recipient registers later, linking happens then.
4. The sender posts a ZAR amount (`POST /remittances`). The API fetches USD/ZAR, applies the fee model, checks daily and monthly limits, and returns a quote with `tracking_ref` (MG plus ten digits) and `expires_at` (15 minutes).
5. The sender may cancel an unexpired quote (`POST /remittances/{id}/cancel`) or let it expire. Cancelled and expired quotes do not consume limits.
6. The sender initiates simulated cash-in (agent cash, bank transfer, or card). An admin confirms receipt. Settlement is not queued before that confirmation.
7. A Redis Streams worker submits an XRPL Testnet Payment from the platform treasury to the recipient’s custodial account, stores the hash, and credits the internal spendable balance.
8. The recipient opens `GET /wallet/me`, sees spendable UCTUSD, incoming transfers with hashes, and cash-out history.

9. The recipient requests a cash-out to USD or ZAR. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). `on_chain_balance` excludes completed cash-outs. Failed cash-outs refund spendable balance.

Anyone with a tracking reference who is the sender, the linked recipient, or an admin can open `/app/track/{ref}` or call `GET /remittances/track/{ref}`.

3 Functional requirements

Table 1 maps the brief’s required behaviour to *our* endpoints. Interactive OpenAPI lives at `/docs`. `GET /health` is unauthenticated.

Table 1: Functional behaviour mapped to this fork’s API.

Capability	Our implementation
Register, login, logout, profile	<code>POST /auth/register</code> , <code>/auth/login</code> , <code>/auth/logout</code> ; <code>GET/PATCH /users/me</code> . Passwords bcrypt; sessions hashed at rest.
Mock KYC + admin review	<code>POST /kyc</code> ; <code>GET /kyc/me</code> , <code>/kyc/me/status</code> ; admin <code>GET /kyc</code> , <code>POST /kyc/{id}/approve reject</code> . ID number encrypted with the KYC Fernet key.
Beneficiaries	<code>POST/GET /beneficiaries</code> . Auto-link on mobile/email.
Quote, fees, limits	<code>POST /remittances</code> ; <code>GET /limits/me</code> ; admin <code>GET/PUT /admin/fee-config</code> , <code>GET/PUT /admin/limit-tiers/{tier}</code> .
Quote lifecycle	TTL 15 minutes; <code>POST /remittances/{id}/cancel</code> ; cash-in rejected if expired or cancelled; fee-consumes-send rejected with 422.
Tracking	<code>tracking_ref</code> on <code>RemittanceOut</code> ; <code>GET /remittances/track/{ref}</code> .
Simulated cash-in	<code>POST /remittances/{id}/cash-in</code> ; admin <code>POST .../confirm-cash-in</code> enqueues settlement.
Queued XRPL settlement	Redis Stream <code>settlement_messages</code> ; <code>POST /admin/settlement/run</code> ; <code>GET /admin/settlement</code> ; <code>POST /admin/settlement/{id}/retry</code> (pending, processing, or failed; completed is 409).
Custodial wallet	<code>GET /wallet/me</code> : <code>balance_rlusd</code> = spendable; also <code>spendable_balance</code> and <code>on_chain_balance</code> (excludes completed cash-outs). Admin <code>GET /admin/platform-wallet</code> reports treasury address, TrustLine, and UCTUSD liquidity.
Simulated cash-out	<code>POST /cash-outs</code> ; <code>GET /cash-outs/me</code> ; admin <code>list/approve/complete/fail</code> . USD and ZAR only.
Sender history	<code>GET /remittances/me</code> ; admin <code>GET /remittances</code> .

4 Fee model

Fees are a single admin-editable `FeeConfig` row, not a history table. Percentages are fractions (0.0100 means 1%). Seeded defaults are a R25.00 fixed fee, 1% of send, 2% FX margin, and 1.5% cash-out.

Let `zar` be the send amount, `fixed` and `percentage` the transaction-fee parameters, `midmarket`

the USD/ZAR rate, and `fx_margin` the spread fraction. Then:

$$\begin{aligned}\text{transaction_fee} &= \text{fixed} + \text{zar} \times \text{percentage} \\ \text{effective_rate} &= \text{midmarket} \times (1 + \text{fx_margin}) \\ \text{net_zar} &= \text{zar} - \text{transaction_fee} \\ \text{rlusd} &= \text{net_zar} / \text{effective_rate}\end{aligned}$$

The quote is rejected (HTTP 422) if $\text{rlusd} \leq 0$ (equivalently if net ZAR after the transaction fee is not positive). Transaction fees are rounded to two decimal places; RLUSD amounts to six, half-up. RLUSD is treated as 1:1 with USD.

The FX margin makes the sender’s conversion *worse* than mid-market: more rand per dollar means fewer dollars received. The cash-out percentage shown on the remittance quote is an *estimate*. At cash-out time the fee is reapplied to the RLUSD amount, USD is 1:1 after that fee, and ZAR cash-out uses the live (or fallback) USD/ZAR rate *without* the FX margin.

4.1 Worked example

Using seeded defaults and the configured fallback mid-market rate of 18.50 (the live API may differ; the algebra does not). For a R1,370.00 send, comparable to the RPW send size discussed by Rateweb (2026):

$$\begin{aligned}\text{transaction_fee} &= 25.00 + 1370.00 \times 0.01 = 38.70 \\ \text{effective_rate} &= 18.50 \times 1.02 = 18.87 \\ \text{net_zar} &= 1370.00 - 38.70 = 1331.30 \\ \text{rlusd} &= 1331.30 / 18.87 = 70.551139\end{aligned}$$

Estimated cash-out fee = $70.551139 \times 0.015 = 1.058267$ RLUSD; estimated recipient payout = 69.492872 RLUSD. The transaction fee (R38.70) is far below the R214–R217 Western Union / MoneyGram flat fees on that send size (Rateweb, 2026). That comparison is indicative only: our cash legs are simulated, we are not an agent network, and a live mid-market rate plus a 2% margin is not a licensed tariff.

A send whose fee consumes the principal is rejected. At the same defaults, R25.00 yields a R25.25 fee and a negative net, so the API returns 422.

Table 2 is the seeded `FeeConfig` an admin can edit at PUT `/admin/fee-config`. There is no fee-history table: the live row is what the next quote uses.

Table 2: Seeded fee parameters (fractions, not whole percents).

Field	Meaning	Default
<code>fixed_fee_zar</code>	Rand added to every send	25.00
<code>percentage_fee</code>	Share of send ZAR	0.0100
<code>fx_margin_percentage</code>	Added to mid-market rate	0.0200
<code>cash_out_fee_percentage</code>	Share of RLUSD at cash-out	0.0150

5 Exchange rates

The brief allows a public API, a mock service, or a configured table (University of Cape Town, 2026a). We use the first, with the third as fallback. GET `https://open.er-api.com/v6/latest/USD` supplies `rates.ZAR`. Successes are cached for 300 seconds (`EXCHANGE_RATE_CACHE_SECONDS`) so concurrent quotes do not stampede a free endpoint. Any failure (network, timeout, non-2xx, missing ZAR field) uses `USD_ZAR_RATE` from `.env` (default 18.50). There is no admin API for the fallback; changing it means editing the environment and restarting, and it only matters while the live API is down. Tests mock the live call except for dedicated exchange-rate tests.

6 Remittance limits

Tiers are derived from KYC, not stored on the user: approved → **verified**, otherwise **unverified**. Seeded caps match the brief's example: unverified R0 / R0 per day and month; verified R3,000 / R25,000. Admins may change caps via PUT `/admin/limit-tiers/{tier_key}`. Unverified is not reachable through POST `/remittances` because that route requires approved KYC; the zero caps still exist so the example table is configurable.

Usage sums the sender's ZAR since the start of the UTC day or calendar month, with two exclusions that the original sketch did not have: **cancelled** never counts; **quoted** rows with **expires_at** already in the past never count. Unexpired quotes and every later live status—including **settlement_failed**—do count. That is deliberate: a failed on-chain payment still represents a confirmed cash-in that occupied limit capacity until an operator decides otherwise.

Table 3: Remittance statuses on one evolving row.

Status	How we get there	Counts to limit?
quoted	POST <code>/remittances</code>	Yes, until TTL
cancelled	sender cancel, still unexpired	No
quoted past TTL	clock; cash-in 409	No
cash_in_pending	sender cash-in	Yes
cash_in_confirmed	admin confirm	Yes
settlement_queued	enqueue after confirm	Yes
settled	worker Payment + credit	Yes
settlement_failed	worker error; no credit	Yes

7 Architecture

Figure 1 is the runtime. The Vite React app on 5173 (or 5174 if 5173 is busy) is the marked web UI. FastAPI serves JSON. SQLAlchemy talks to SQLite through Alembic revisions 0001_initial, 0002_quote_session, and 0003_cash_out_burn. Redis is transport for settlement, not the source of truth. The settlement worker (the same function as POST `/admin/settlement/run` and `python -m scripts.run_settlement_worker`) signs Testnet Payments with `xrpl-py`. CORS allows the local Vite and API origins, including loopback.

Figure 2 is the money path the brief asks for.

Wallet design is a hybrid of the brief's two options (University of Cape Town, 2026a,b). One `PlatformWallet` holds scarce IOU liquidity. Each recipient still receives a real custodial Testnet

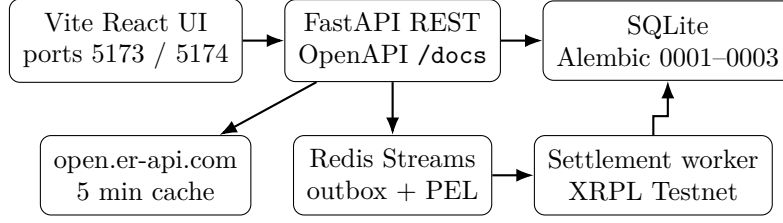


Figure 1: Main components. Redis carries settlement messages; durable status lives in SQLite.

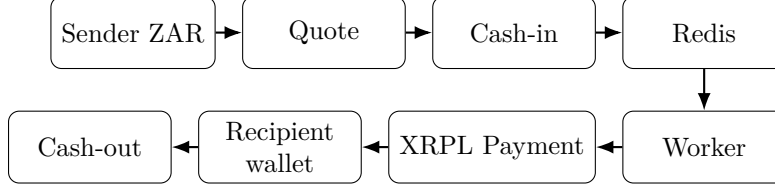


Figure 2: Settlement path: confirmed rand in, queued Payment, custodial UCTUSD, simulated fiat out.

account, generated lazily on first settlement, so FR-style “individually attributable Payment + hash” is a genuine ledger transaction rather than a book entry that pretends to be one. Address and encrypted secret are committed *before* TrustSet. If an address exists but the trust line is not yet established, the worker reuses the stored secret; it does not mint a second wallet.

XRP is the only asset an XRPL account holds natively; UCTUSD and RLUSD are issued tokens (IOUs). A holder can accept an IOU only after a TrustSet opens a trust line to the issuer, which reserves a little XRP (locked, not spent) (University of Cape Town, 2026b). Rippling lets the same issued token move between holders of that issuer without an explicit hop back through the issuer: Default Ripple is normally on for the issuer and off for ordinary holders. We do not implement rippling ourselves; settlement is a direct treasury-to-recipient Payment after TrustSet.

8 Cash-in, UCTUSD settlement, and cash-out

8.1 Cash-in

`POST /remittances/{id}/cash-in` is allowed only on an unexpired, non-cancelled `quoted` remittance. The sender chooses `agent_cash`, `bank_transfer`, or `card`. Status becomes `cash_in_pending`. Admin `confirm-cash-in` sets `cash_in_confirmed` and calls `enqueue_settlement`. No Payment is submitted at this step. The cash movement itself is simulated: there is no card acquirer, no agent till, and no bank confirmation webhook.

8.2 UCTUSD settlement

Enqueue inserts at most one `SettlementMessage` per remittance (unique constraint) and `XADDs` to stream `settlement_messages`. The Redis Stream ID is stored on `stream_entry_id`. If the database row exists but the stream ID is missing or the entry is gone, a later enqueue *republishes*—that covers “DB commit succeeded, Redis `XADD` failed”. Redis is transport; the database row is source of truth.

The worker first reclaims its pending-entries list (`XREADGROUP` id 0), then idle pending entries (`XAUTOCLAIM` on Redis 6.2+, else `XPENDING+XCLAIM`; local Memurai 5.0.14 has no `XAUTOCLAIM`), then

new > entries. Entries are acknowledged only after the message is **completed** or **failed**. A crash mid-flight leaves the entry in the PEL for reclaim.

Before submitting a Payment, if `xrpl_settlement_tx_hash` is already set, the worker marks completed and **does not pay or credit again**. On the success path the Payment memo carries the remittance id; the hash, recipient credit, **settled**, and **completed** are one database commit. Admin retry accepts **failed**, **processing**, and **pending** (reset to pending, republish). **completed** returns 409.

8.3 Cash-out

The recipient must have approved KYC to cash out but may hold UCTUSD without it. Spendable `RecipientWallet.balance` is debited atomically at request time (`UPDATE ...WHERE balance >=:amt`). `GET /wallet/me` reports `balance_rlsud` and `spendable_balance` as that spendable figure (the slot is currently UCTUSD; official RLUSD later via `.env` if funded). `on_chain_balance` is spendable plus cash-out amounts in **requested** or **approved** only; it excludes completed cash-outs. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). After a request, spendable drops while on-chain stays higher by the reserved amount until complete, when the on-chain figure drops too. A failed cash-out refunds spendable.

9 Database

Runtime schema is Alembic, not `create_all` alone. FastAPI lifespan runs `alembic upgrade head` then seeds default fees and limit tiers. Tests still use in-memory `create_all`. Figure 3 is the entity–relationship view; the list below is the column notes.

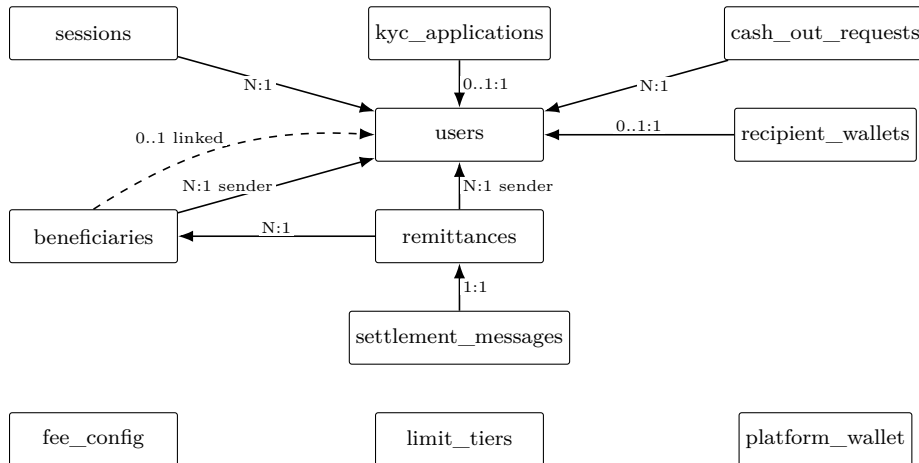


Figure 3: Entity–relationship view of the Alembic schema. Cardinalities are child:parent. The dashed edge is optional `linked_user_id`. Cash-out is not foreign-keyed to a remittance: payouts draw aggregated spendable UCTUSD. `fee_config` and `platform_wallet` are singletons; `limit_tiers` is the unverified/verified lookup.

- **users** — name, unique email and mobile, bcrypt `password_hash`, role.
- **sessions** — `token_hash` (SHA-256 hex, unique), expiry, `revoked_at`. No plaintext token column.

- `kyc_applications` — one row per user; identification number encrypted.
- `beneficiaries` — payout profile; optional `linked_user_id`.
- `remittances` — full lifecycle on one row: amounts, fees, status (including `cancelled`), `tracking_ref`, `expires_at`, cash-in fields, XRPL hash.
- `settlement_messages` — one per remittance; status; `stream_entry_id`; attempts; failure reason.
- `recipient_wallets` — custodial address, encrypted secret, trust-line flag, spendable `balance`. No `on_chain_balance` column; that figure is computed on read.
- `platform_wallet` — singleton treasury address and encrypted secret.
- `cash_out_requests` — RLUSD amount, fiat, fee, payout, status, admin action stamps.
- `fee_config`, `limit_tiers` — singleton fees; unverified/verified caps.

Encrypted columns are stored as `VARCHAR(500)` (Fernet ciphertext). Existing SQLite files created only with `create_all` have no `alembic_version` table and will fail upgrade; demo databases should start empty and migrate.

10 API overview

The API is JSON over HTTP with Bearer sessions. FastAPI emits Swagger UI at <http://127.0.0.1:8000/docs> and the OpenAPI schema at `/openapi.json`. Route prefixes: `auth`, `users`, `kyc`, `beneficiaries`, `remittances`, `limits`, `wallet`, `cash-outs`, and `admin` (fees, limits, settlement, platform wallet). CORS defaults allow Vite on 5173 and 5174 (`localhost` and `127.0.0.1`), local React on 3000, and the API host on 8000, overridable with `CORS_ORIGINS` as a JSON list. Credentials are allowed. The SPA stores the bearer token in `sessionStorage` and sends it as `Authorization: Bearer`.

Authz in short: customers act on their own resources; KYC-approved customers create quotes, beneficiaries, cash-in, and cash-outs; admins review KYC, confirm cash-in, run and retry settlement, action cash-outs, and edit fees and tiers. Tracking is visible to the sender, the linked recipient, or an admin (403 otherwise; 404 if the ref does not exist).

11 Security

Passwords are bcrypt via `passlib` and never stored in plaintext. Login returns a urlsafe bearer token once; the database stores only `SHA-256` of that token. Logout sets `revoked_at`. Opaque sessions were chosen over JWT because this is a single FastAPI process: logout must actually revoke access, which a signed JWT cannot do without a server-side denylist that looks like the sessions table anyway.

Two Fernet keys are required: `KYC_ENCRYPTION_KEY` for the identification number, `XRPL_KEY_ENCRYPTION_KEY` for platform and recipient XRPL secrets. A leak of one category does not decrypt the other. Keys live in environment variables, not in the database, not in API responses, not in application logs, and not in git (`.env` is local; `.env.example` has placeholders). XRPL secrets are decrypted only in the signing path. Bad ciphertext raises `RuntimeError` rather

than returning empty values. Private keys are never printed by `setup_platform_wallet`. There is no custom application logger beyond uvicorn's access log (method/path/status, no bodies).

CORS is a demo convenience for the local Vite origin, not an authentication mechanism. Production would restrict origins, put keys in a secrets manager, rotate Fernet keys with a dual-decrypt window, and stop using SQLite as the vault for encrypted seeds.

12 Regulatory considerations

This section is an understanding of the problem, not a legal opinion (University of Cape Town, 2026a). A Testnet IOU does not make the operator a licensed money transmitter, and it does not take the corridor outside FICA or SARB rules.

KYC and AML. South African accountable institutions under FICA must identify customers and understand source of funds before providing money-remittance services. Our mock KYC collects the brief's fields, encrypts the ID number, and gates sending and cash-out on admin approval. That is a teaching model of a control, not a CDD programme: we do not screen against sanctions lists, we do not verify documents against Home Affairs, and we do not file suspicious-transaction reports.

Transaction monitoring (not fully automated). A real remittance business is expected to monitor velocity, structuring, mule patterns, and corridor risk. We enforce daily and monthly ZAR caps and keep a full remittance and settlement audit trail, including XRPL hashes. We do *not* ship an AML product: no rules engine, no alert queue, no SAR workflow. Operators would still need people and systems we have not built.

Custody of crypto assets. Recipient wallets are custodial: the platform generates Testnet keys, encrypts them, and signs on the customer's behalf. That is convenient and it is also a custody business. In production that would engage crypto-asset service-provider expectations (safekeeping, segregation, operational resilience) under the evolving South African CASP / FSCA framework. We store keys encrypted with a dedicated Fernet key; we do not offer self-custody withdrawal of the secret.

SARB and exchange control. Cross-border rand payments sit inside South African exchange-control and authorised-dealer arrangements. A stablecoin hop does not create a new legal corridor. Our cash-in is simulated, so we never actually take rand across the border. A live system would still need an authorised channel for the fiat legs and would still need to report what the SARB requires. RLUSD on Testnet is not a SARB-approved mechanism for discharging an exchange-control obligation.

Licensing. A commercial Western Union competitor in South Africa typically needs a FICA accountable-institution posture, Reserve Bank authorisation where the activity is a remittance or payment service, and—if offering crypto custody or exchange—the applicable FSCA crypto-asset authorisation. Agent cash-in/out may also engage payment-system and consumer-protection rules (clear fees, receipts, complaint handling). This coursework prototype is not licensed and must not be pointed at real customer funds or Mainnet credentials (University of Cape Town, 2026a).

Consumer protection and safeguarding follow from the same honesty: quoted fees should match charged fees (our quote locks the breakdown for 15 minutes); customer secrets should not leak (Section 11); and simulated balances must not be described as redeemable bank money. We do not hold client fiat in a trust account because we never collect real fiat. The web UI in Section 15 shows the locked fee breakdown, the MG tracking reference, and an academic contact block (course, group, campus, repository). That is coursework disclosure, not a licensed complaints desk.

13 Assumptions and limitations

Assumptions we adopted on purpose: one customer identity for send and receive; admins only via `scripts/create_admin.py`; one KYC row with overwrite-on-resubmit until approved; beneficiary auto-match rather than invite-to-register; hybrid treasury plus per-recipient XRPL accounts; issuer address and currency code as `.env` so UCTUSD versus RLUSD is a config change (University of Cape Town, 2026b); Redis Streams as the queue; SQLite for development; quote TTL of 900 seconds.

Remaining limitations of *this* fork, not of the old sketch:

- SQLite’s single-writer model is not a production remittance ledger.
- Cash-in and fiat cash-out are simulated; complete still burns reserved UCTUSD on-chain by paying the issuer.
- Automated tests mock XRPL (faucet, TrustSet, Payment). Redis is real (DB 15).
- Redis is required at runtime and in pytest.
- Login latency is dominated by bcrypt (see Section 14).
- No AML monitoring product, no sanctions screening, no production key-management service.
- Daily/monthly limit checks and the quote insert are separate steps with no row lock between them; two concurrent quote requests from the same sender could both read usage under the limit and jointly exceed it. Acceptable for a single-process academic prototype, not for a production ledger.

14 Performance summary

HTTP load, queue enqueue, and five live Testnet Payments were re-run on this fork on 7 September 2026 (50 concurrent synthetic users, 60 s, Windows, single-process uvicorn and SQLite). The tables and charts live in `PERFORMANCE_TESTING.md` and `backend/perf/results/`.

The HTTP mix delivered 2,218 requests, zero failures, about 37 req/s. Authenticated reads sat around 20–33 ms at the median; `POST /remittances` about 66 ms median. `POST /auth/login` remains the outlier (about 850 ms median) because bcrypt is deliberately expensive. Queue enqueue on this OneDrive-backed SQLite host measured about 29 messages/s (a 1 September Mac sample of the same Redis path was 517 msg/s). Five real Testnet Payments completed 5/5 and averaged about 17.5 s each—ledger close time, not application CPU (a 1 September sample of the same script was 12.4 s/tx). Relative bottlenecks—bcrypt on login, consensus on settlement—are unchanged in kind.

15 Web UI

The marked web application is **frontend/**: Vite, React, TypeScript, and React Router. It calls the FastAPI JSON API with `VITE_API_URL` (default <http://127.0.0.1:8000>). The landing page is a marketing scroll (hero, quote explainer as `#faq`, beneficiary/scan/methods, contact as `#contact`, footer). Login, register, and the authenticated shells use a DASH-style grid-trail background. Landing motion is GSAP ScrollTrigger plus original Three.js objects—not third-party brand assets.

Table 4 is the shipped screen map. The quote screen shows send amount, locked transaction fee, net ZAR after that fee, FX margin, rate, and UCTUSD. Empty states are implemented, not planned: unverified senders see KYC pending, not a send form; expired quotes show expired and hide cash-in; a recipient without a Testnet account yet still sees a zero spendable wallet row; settlement failure shows the worker reason (for example `tecPATH_DRY`) rather than a dummy hash.

Table 4: Shipped screens mapped to routes and endpoints.

Screen	Route	Endpoints
Landing / FAQ / Contact	<code>/</code> , <code>#faq</code> , <code>#contact</code>	static; Contact states course, group, campus, GitHub
Register / login	<code>/register</code> , <code>/login</code>	POST <code>/auth/register</code> , <code>/login</code> , <code>/logout</code>
Sender home	<code>/app</code>	GET <code>/users/me</code> , <code>/kyc/me/status</code> , <code>/limits/me</code>
Profile	<code>/app/profile</code>	GET/PATCH <code>/users/me</code>
KYC form	<code>/app/kyc</code>	POST <code>/kyc</code> , GET <code>/kyc/me</code>
Beneficiaries	<code>/app/beneficiaries</code>	GET/POST <code>/beneficiaries</code>
New quote / quote	<code>/app/send</code> , <code>/app/send/{id}</code>	POST <code>/remittances</code> , <code>.../cancel</code> , <code>.../cash-in</code>
History / track	<code>/app/history</code> , <code>/app/track/{ref}</code>	GET <code>/remittances/me</code> , <code>/remittances/track/{ref}</code>
Recipient wallet	<code>/app/wallet</code>	GET <code>/wallet/me</code>
Cash-out	<code>/app/cash-out</code>	POST <code>/cash-outs</code> , GET <code>/cash-outs/me</code>
Admin KYC	<code>/admin/kyc</code>	GET <code>/kyc</code> , POST <code>.../approve reject</code>
Admin cash-in	<code>/admin/cash-in</code>	GET <code>/remittances</code> , POST <code>.../confirm-cash-in</code>
Admin settlement	<code>/admin/settlement</code>	GET <code>/admin/settlement</code> , POST <code>.../run</code> , <code>.../{id}/retry</code>
Admin cash-out	<code>/admin/cash-out</code>	GET <code>/cash-outs</code> , POST <code>.../approve complete fail</code>
Admin config	<code>/admin/config</code>	GET/PUT <code>/admin/fee-config</code> , <code>/admin/limit-tiers</code>
Platform wallet	<code>/admin/platform-wallet</code>	GET <code>/admin/platform-wallet</code> ; TrustLine helper

Admins are created with `scripts/create_admin.py`, never via public register. The examiner demo is the UI at <http://127.0.0.1:5173/> (or 5174) against the API on 8000: register or sign in, KYC, beneficiary, quote, cash-in, admin confirm, settlement run or retry, track, then recipient wallet and optional cash-out. Swagger at `/docs` remains a developer tool, not the marked product.

References

FinMark Trust (2024). South africa to the rest of SADC remittances market assessment 2024. Technical report, FinMark Trust.

International Monetary Fund (2025). South africa: Technical assistance report—cross-border payments: South africa–zimbabwe corridor diagnostic. Technical Report 25/93, International Monetary Fund.

Rateweb (2026). Your transfer fee ignores where you’re sending. that costs you. Online article analysing World Bank Remittance Prices Worldwide 2025 Q1 figures for South African outbound corridors. Secondary citation of World Bank RPW 2025 Q1. Reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed R1,370 send.

University of Cape Town (2026a). ECO5040W class project brief: XRPL-based FX remittance platform using RLUSD. Course brief, Financial Software Engineering.

University of Cape Town (2026b). ECO5040W project resources. Trust lines, IOUs, RLUSD vs fallback IOU, recommended stack.

World Bank (2025). Remittance prices worldwide. Technical report, World Bank. Quarterly survey of the cost of sending remittances. Primary source for corridor fee comparisons.